

Part III: Docker Compose – Multi-Container Deployment

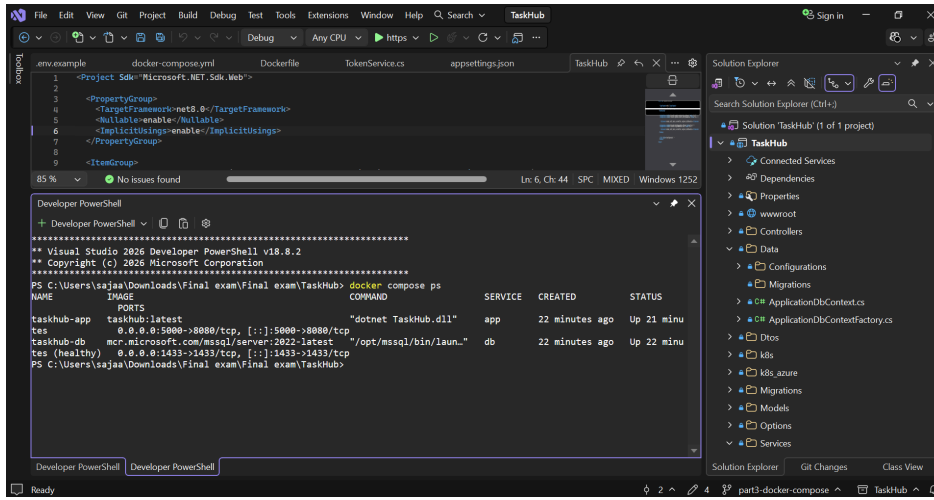


Figure 1: Docker Compose Services Running Successfully

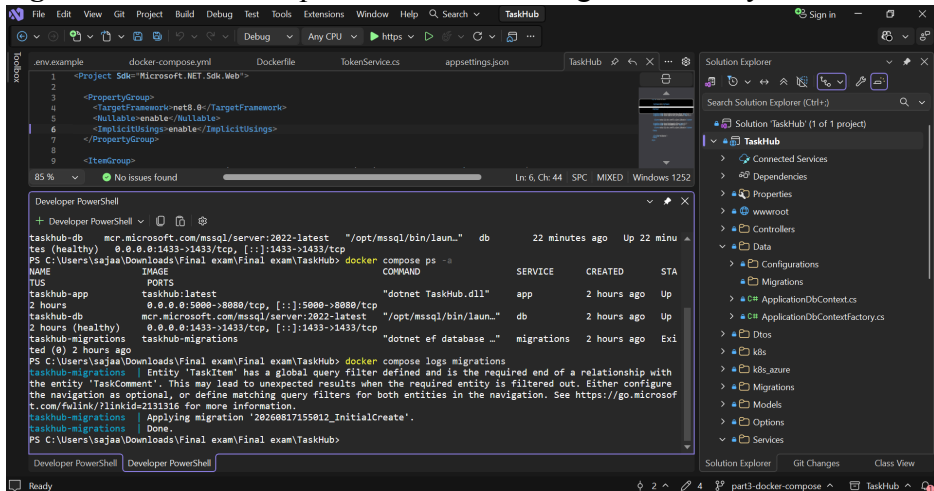


Figure 2: Database Migration Completed Successfully

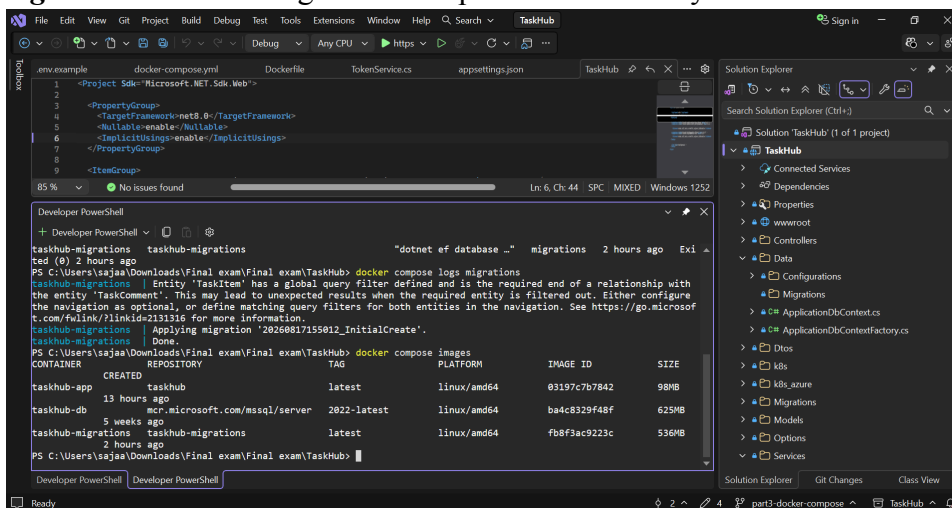


Figure 3: Docker Compose Images for TaskHub Services

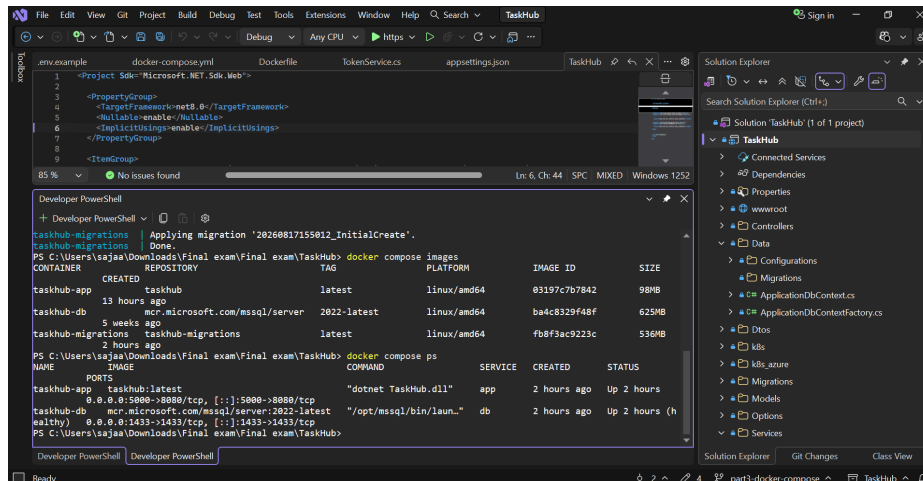


Figure 4: Docker Compose Application and Database Container Status

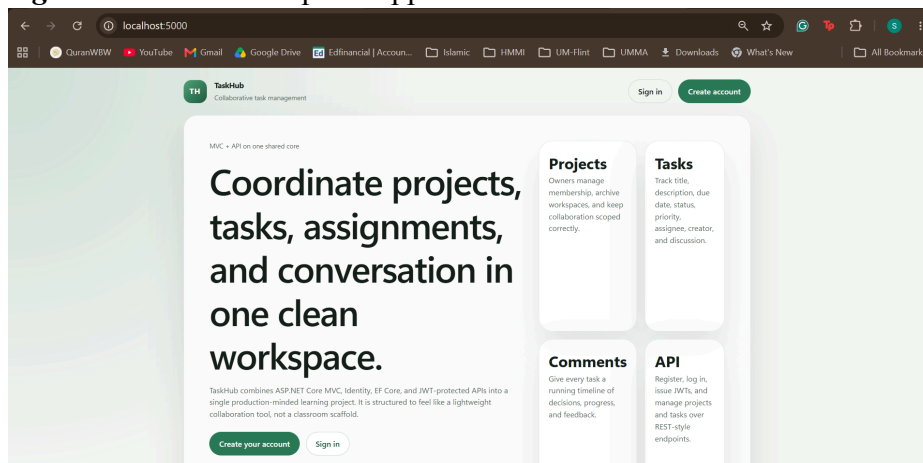


Figure 5: TaskHub Application Running on localhost:5000

What Changed and What Worked

I completed the Docker Compose configuration for the SQL Server, migration, and TaskHub application services. I configured the required environment variables, port mappings, service dependencies, health check, and restart behavior. After the configuration was validated, the environment built and started successfully. SQL Server reported a healthy status, the migration completed successfully, and the TaskHub application was accessible at localhost:5000.

Issues Encountered and Resolution

While configuring the Docker Compose environment, I encountered YAML formatting and indentation errors in the docker-compose.yml file that initially prevented Docker Compose from validating the configuration. I corrected the service indentation and structure and used docker compose config to verify the configuration. I also had to make sure the required environment variables were properly set before starting the containers. After making these corrections, the configuration validated successfully and the environment started as expected.

1. Why are health checks useful?
 - a. Health checks are useful because they verify that a service is actually ready and functioning, rather than just checking whether the container is running. In my setup, the SQL Server health check helped make sure the database was ready before the dependent services tried to use it.
2. Why is startup order important?
 - a. Startup order is important because some services depend on other services being available first. In this setup, the application and migration service depend on SQL Server. If they start before the database is ready, the database connection or migrations could fail.
3. Why do long-running services use restart policies?
 - a. Long-running services use restart policies so they can automatically recover if they stop or fail unexpectedly. For services such as the web application and database, this helps keep the environment available without requiring the containers to be restarted manually.